

TECHNICAL PRODUCTION MANUAL

Project: Industrial-Grade Smart Bathroom Presence Sensor

Core Architecture: Dual-Stack Redundancy (Local Failsafe + LAN MQTT + Peer-to-Peer ESP-NOW) **Target Microcontroller:** ESP32-C3 SuperMini **Sensor Variant:** HLK-LD2410B (24GHz FMCW mmWave Radar)

1. Complete Bill of Materials (BoM) & Budget

This component list is optimized to satisfy industrial ruggedness and high-voltage grid transient safety for environments like humid bathrooms.

Component	Industry Specification / Rating	Est. Price (BDT)
ESP32-C3 SuperMini	RISC-V 32-bit single-core MCU, 2.4 GHz Wi-Fi/BLE	220 BDT
HLK-LD2410B	24GHz FMCW radar module with Bluetooth configuration	360 BDT
HLK-PM01	Isolated AC-DC Step-Down Module (Input: 100-240V AC, Output: 5V DC / 3W)	160 BDT
5V Single Relay Module	Opto-isolated high-current switch (Rating: 10A 250V AC)	100 BDT
10D471K Varistor (MOV)	Metal Oxide Varistor, 470V clamping threshold for surge protection	15 BDT
Glass Fuse & Holder	Fast-acting cylindrical glass fuse, rated at 1A 250V AC	15 BDT
Passive Elements	Resistors (10kΩ pull-up), prototype PCB, and terminal blocks	50 BDT
Total Estimated Cost		~920 BDT

2. Circuit Architecture & High-Voltage Isolation Rules

High-Voltage Mains Protection (AC Side)

To safeguard low-voltage microcontrollers against grid switching transients, lightning spikes, and catastrophic shorts within a residential wiring grid, you must implement a strict protective input layer.

- The Failsafe Sequence:** Mains Live (220V AC) must route through the 1A Glass Fuse first. The 10D471K Varistor (MOV) must be placed in parallel *after* the fuse, directly bridging the fused Live line and the Neutral line.
- Overvoltage Execution:** If an industrial voltage spike or surge exceeds 470V, the MOV's

internal resistance collapses instantly to near zero ohms. This creates an intentional, localized short circuit that forces current to spike rapidly, instantly blowing the 1A glass fuse and isolating the down-line electronics from the grid before damage occurs.

Low-Voltage Signal Routing & Isolation (DC Side)

- **Creepage & Clearance:** Keep a physical distance of at least 6mm on your prototyping board between any high-voltage track (220V AC) and low-voltage logic track (5V/3.3V DC).
- **Opto-Isolation:** Use a relay module that features onboard optocoupler isolation. This isolates the inductive kickback and electromagnetic noise of the switching relay coil away from the ESP32-C3 GPIO pins.

3. Detailed Pinout Interconnections

Power Distribution Loop

1. **Mains 220V Live (L)** \rightarrow Terminal 1 of 1A Fuse Holder.
2. **Fuse Holder Terminal 2** \rightarrow Connected to MOV Pin 1 **AND** HLK-PM01 AC Input Pin (L).
3. **Mains 220V Neutral (N)** \rightarrow Connected to MOV Pin 2 **AND** HLK-PM01 AC Input Pin (N).
4. **HLK-PM01 DC Output (+5V)** \rightarrow Connects to the +5V Power Rail (Feeds ESP32-C3 5V pin, Relay VCC, and LD2410B VCC).
5. **HLK-PM01 DC Output (GND)** \rightarrow Connects to the Common Ground Rail (Feeds ESP32-C3 GND, Relay GND, and LD2410B GND).

Logic Control Loop

- **LD2410B OUT Pin** \rightarrow Connects to ESP32-C3 **GPIO 8**.
- **Relay Module IN Pin** \rightarrow Connects to ESP32-C3 **GPIO 10**.

4. Primary Node Production Firmware (ESP32-C3)

This firmware uses a strictly non-blocking asynchronous architecture. The execution of the localized radar sensor and the light relay is independent of network status. If Wi-Fi breaks or the MQTT broker drops offline, the hardware execution loop continues to function instantly.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <esp_now.h>

// Pin Configuration
#define RADAR_PIN 8
#define RELAY_PIN 10

// Network & Broker Credentials
const char* ssid = "YOUR_WIFI_SSID";
```

```

const char* password = "YOUR_WIFI_PASSWORD";
const char* mqtt_server = "192.168.1.100"; // Replace with your
Raspberry Pi IP

// Peer MAC Address (Master ESP32-WROOM)
uint8_t masterMacAddress[] = {0x24, 0x0A, 0xC4, 0xFF, 0xFF, 0xFF}; //
Replace with actual MAC

// Class Instances
WiFiClient espClient;
PubSubClient mqtt(espClient);

// Timing and System State Monitoring
bool lightState = false;
unsigned long lastPresenceTime = 0;
const unsigned long OFF_DELAY_MS = 3000; // Hardcoded 3-second timeout
execution
unsigned long lastNetworkCheck = 0;

// Data Structure for P2P Broadcast
typedef struct struct_message {
    char nodeName[16];
    bool currentLightState;
} struct_message;
struct_message outboundData;

void transmitTelemetry(bool state) {
    outboundData.currentLightState = state;

    // Broadcast peer-to-peer over ESP-NOW
    esp_now_send(masterMacAddress, (uint8_t *) &outboundData,
sizeof(outboundData));

    // Publish over LAN MQTT network if active
    if (mqtt.connected()) {
        if (state) {
            mqtt.publish("home/bathroom/light/state", "ON", true);
        } else {
            mqtt.publish("home/bathroom/light/state", "OFF", true);
        }
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(RADAR_PIN, INPUT);
    pinMode(RELAY_PIN, OUTPUT);
}

```

```

digitalWrite(RELAY_PIN, LOW); // Failsafe state: Light starts OFF

// Initialize Wi-Fi Layer in Station Mode
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

// Initialize ESP-NOW Protocol Layer
if (esp_now_init() == ESP_OK) {
    esp_now_peer_info_t peerInfo = {};
    memcpy(peerInfo.peer_addr, masterMacAddress, 6);
    peerInfo.channel = 0;
    peerInfo.encrypt = false;
    esp_now_add_peer(&peerInfo);
}

mqtt.setServer(mqtt_server, 1883);
strcpy(outboundData.nodeName, "BATH_SENS_C3");
}

void loop() {
    // --- STEP 1: CRITICAL LOCAL HARDWARE CONTROL ---
    bool humanPresent = (digitalRead(RADAR_PIN) == HIGH);

    if (humanPresent) {
        lastPresenceTime = millis(); // Refresh timestamp continuously
    while present
        if (!lightState) {
            digitalWrite(RELAY_PIN, HIGH); // Instantaneous light
activation
            lightState = true;
            transmitTelemetry(true);
        }
    } else {
        // Enforce a strict 3-second delay after presence signature
drops
        if (lightState && (millis() - lastPresenceTime >=
OFF_DELAY_MS)) {
            digitalWrite(RELAY_PIN, LOW); // De-energize relay
instantly
            lightState = false;
            transmitTelemetry(false);
        }
    }

    // --- STEP 2: ASYNCHRONOUS BACKGROUND NETWORK HANDLING ---
    if (WiFi.status() == WL_CONNECTED) {
        if (!mqtt.connected()) {
            if (millis() - lastNetworkCheck > 5000) { // Limit retry

```

```

connection requests to 5s intervals
        lastNetworkCheck = millis();
        if (mqtt.connect("ESP32_Bathroom_Client")) {
            // Resend current state upon successful
re-connection
            transmitTelemetry(lightState);
        }
    }
} else {
    mqtt.loop();
}
}
}

```

5. Secondary Backup Master Node Firmware (ESP32-WROOM)

This code runs continuously on your central master ESP32-WROOM module to catch peer-to-peer data over ESP-NOW, bypassing the local router entirely.

```

#include <WiFi.h>
#include <esp_now.h>

typedef struct struct_message {
    char nodeName[16];
    bool currentLightState;
} struct_message;
struct_message incomingData;

void onDataPacketReceived(const uint8_t * mac, const uint8_t
*incomingDataPtr, int len) {
    memcpy(&incomingData, incomingDataPtr, sizeof(incomingData));

    Serial.print("[ESP-NOW BACKUP ALIVE] Node Source: ");
    Serial.print(incomingData.nodeName);
    Serial.print(" | Light State Output: ");
    Serial.println(incomingData.currentLightState ? "ON" : "OFF");
}

void setup() {
    Serial.begin(115200);

    WiFi.mode(WIFI_STA);
    WiFi.disconnect(); // Do not join router; listen directly on raw
radio frequency channels

    if (esp_now_init() == ESP_OK) {

```

```

        esp_now_register_recv_cb(onDataPacketReceived);
        Serial.println("Master System Ready. Monitoring Backup
Channels.");
    } else {
        ESP.restart();
    }
}

void loop() {
    // Master node process runs exclusively on hardware receive
    interrupts
}

```

6. Home Assistant YAML Local MQTT Integration

Add this configuration directly to your Home Assistant configuration.yaml file on your Raspberry Pi. This maps the background MQTT status topics to a local entity.

```

mqtt:
  binary_sensor:
    - name: "Bathroom Presence Status"
      state_topic: "home/bathroom/light/state"
      payload_on: "ON"
      payload_off: "OFF"
      device_class: occupancy

  light:
    - name: "Bathroom Mirror Light"
      state_topic: "home/bathroom/light/state"
      payload_on: "ON"
      payload_off: "OFF"
      optimistic: false
      retain: true

```